

MiniORM - A Custom Object-Relational Mapping Python Library

DESIGN PATTERNS

NATALIA MARCINKOWSKA, KAROLINA ROZMUS, MAX SŁOTA



Contents

1	MiniORM Library	2
1.1	Library modules	2
1.1.1	base.py	2
1.1.2	orm_types.py	2
1.1.3	states.py	2
1.1.4	mapper.py	2
1.1.5	inheritance.py	3
1.1.6	identity_map.py	3
1.1.7	database.py	3
1.1.8	builder.py	3
1.1.9	generator.py	3
1.1.10	query.py	3
1.1.11	transactions.py	3
1.1.12	session.py	4
1.2	Class diagram	5
2	MiniORM Demo	6
2.1	Physical architecture	6
2.2	Logical architecture	7
3	Implemented patterns	7
3.1	Strategy	7
3.2	Unit of Work	8
3.3	Data Mapper	9
3.4	Identity Map	10
3.5	Association Table	11

1 MiniORM Library

1.1 Library modules

1.1.1 base.py

MiniBase — Model base class. Subclassing triggers `__init_subclass__`, which:

- extracts columns/relationships from class body
- constructs `Mapper`
- registers in `MiniBase._registry`

1.1.2 orm_types.py

AssociationTable — Descriptor for M2M junction table: `name`, `local_key`, `remote_key`, `local_table`, `remote_table`.

Relationship — Declares association between models: `target_table`, `r_type` (many-to-one, one-to-many, many-to-many), `backref`, `cascade_delete`. **Mapper** resolves to FK columns or `AssociationTable`.

Column — Base for typed columns

Text — Column with `dtype=str`.

Number — Column with `dtype=int`.

1.1.3 states.py

ObjectState — Enum:

- `TRANSIENT` (new, not in DB)
- `PENDING` (queued insert)
- `PERSISTENT` (in DB, tracked)
- `EXPIRED` (stale after commit)
- `DELETED` (queued delete)
- `DETACHED` (no session)

1.1.4 mapper.py

Mapper — Maps model class to DB schema. Attributes:

- `cls` — the class being mapped
- `table_name` — name of corresponding table in database
- `columns` — columns held in the corresponding table
- `relationships` — relationships with other classes, represented as foreign keys in the database
- `pk` — primary key of the table
- `inheritance` — the inheritance interface, containing strategy and the discriminator value
- `parent` — the parent mapper in inheritance
- `children` — child mappers in inheritance

Methods:

- `_resolve_parent`
- `_resolve_inheritance`

- `_resolve_tablename`
- `_resolve_columns`
- `_resolve_relationships`
- `_resolve_pk`
- `hydrate`
- `finalizemappers`

1.1.5 inheritance.py

InheritanceStrategy — Abstract base with methods:

- `resolve_columns`
- `resolve_tablename`
- `resolve_insert`
- `resolve_update`
- `resolve_delete`
- `resolve_targetclass`
- `resolve_attributes`

SingleTableInheritance — All subclasses in one table; discriminator column.

ClassTableInheritance — One table per class; JOIN on PK.

ConcreteTableInheritance — One table per concrete class; UNION for polymorphic queries.

Inheritance — Wrapper interface

1.1.6 identity_map.py

IdentityMap — Cache mapping `(model_class, pk) -> instance`. Ensures one Python object per DB row.

1.1.7 database.py

DatabaseEngine — Wraps `sqlite3.connection`.

1.1.8 builder.py

QueryBuilder — Builds SQL strings. Returns `(sql, params)`; does not execute queries.

1.1.9 generator.py

SchemaGenerator — Creates/drops tables; Adds missing columns via ALTER TABLE.

1.1.10 query.py

Query — Created by `session.query(Model)`. Uses mapper to hydrate fetched rows.

1.1.11 transactions.py

Transaction — Abstract base

InsertTransaction — `prepare()` yields `(sql, params, callback, rebuild_fn)` per table; callbacks set PK, add to identity map, flush M2M, take snapshot.

UpdateTransaction — `prepare()` uses mapper's `prepare_update` and builder's `build_update`

DeleteTransaction — `prepare()` uses mapper's `prepare_delete` and builder's `build_delete`.

1.1.12 session.py

Session — Central coordinator. Holds:

- engine
- query_builder
- identity_map
- unit_of_work

Methods:

- add
- update
- delete
- flush
- commit
- rollback
- _take_snapshot

1.2 Class diagram

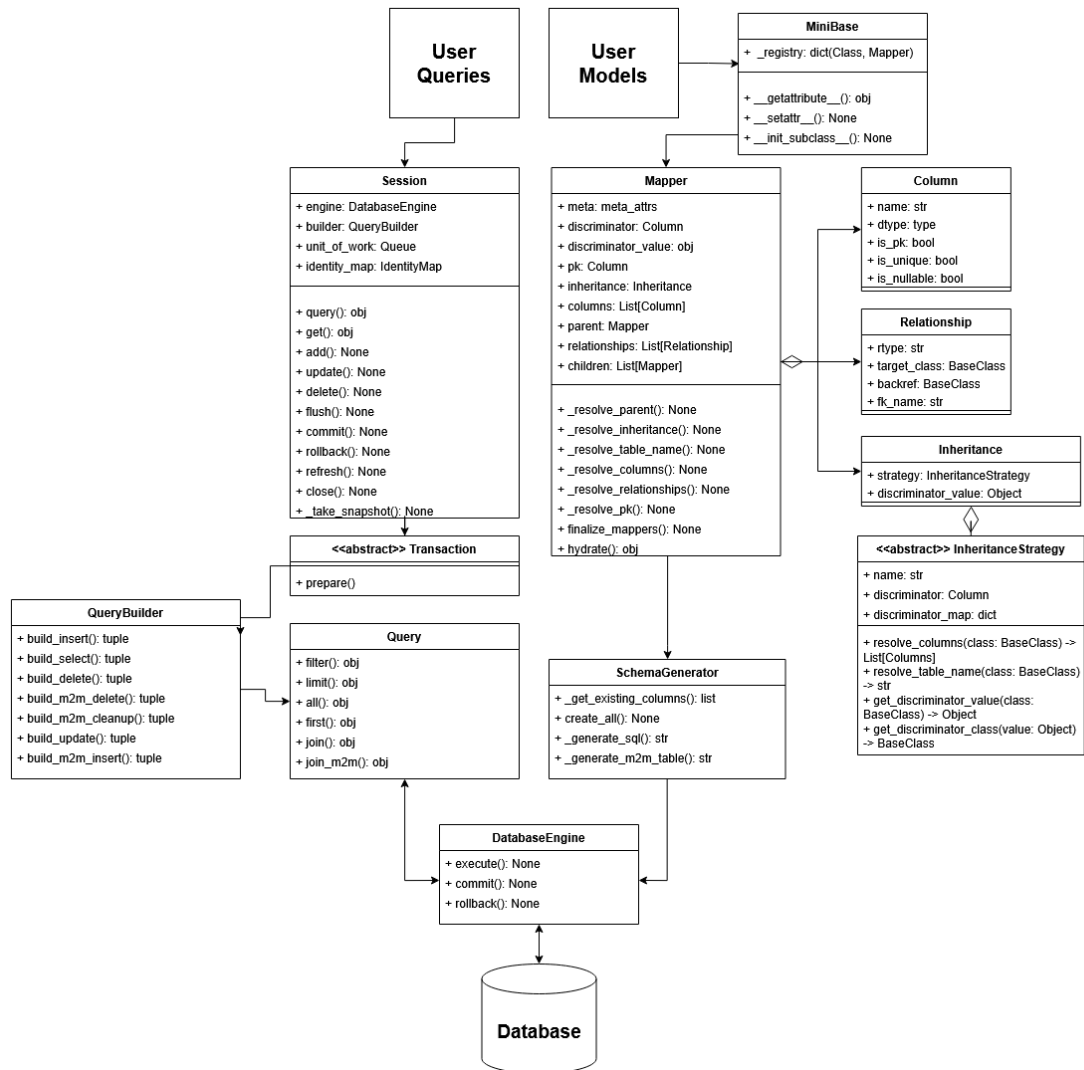


Figure 1: Logical system architecture diagram

2 MiniORM Demo

2.1 Physical architecture

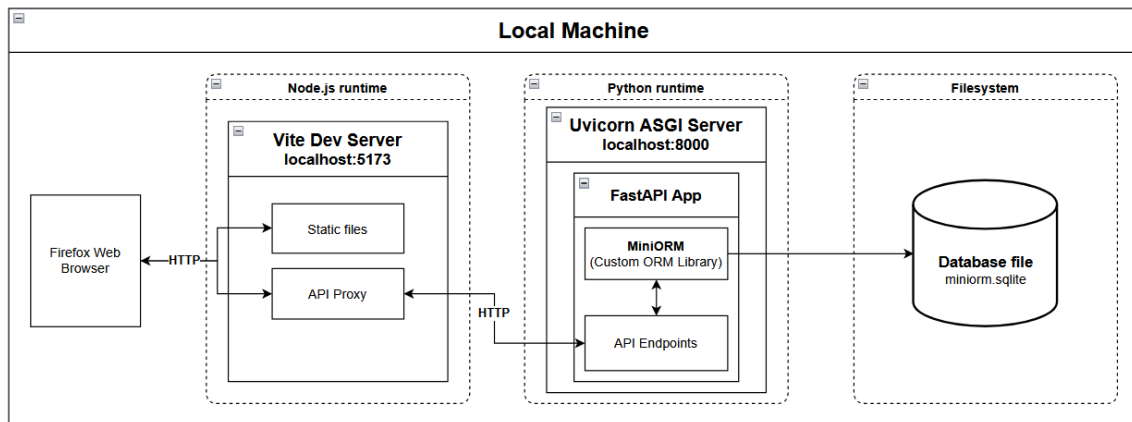


Figure 2: Physical system architecture diagram

Component description:

- **Browser** (React client)
 - Sends HTTP requests for static files to **Vite Dev Server**
 - Sends API requests to **Vite Dev Server** (proxied to FastAPI)
- **Vite Dev Server** (Node.js runtime)
 - Serves React static assets (HTML, JS, CSS)
 - Proxies API requests (`/api/*`) to **FastAPI Backend** over HTTP
- **FastAPI Backend** (Python runtime + MiniORM)
 - Receives proxied HTTP API requests from Vite
 - Handles business logic
 - Uses **Custom ORM** internally to query or update the database
- **MiniORM**
 - Maps Python models to database tables
 - Sends SQL queries over TCP using the PostgreSQL wire protocol
 - Returns query results to FastAPI endpoints
- **PostgreSQL Service** (Database engine + storage)
 - Listens on TCP port 5432
 - Executes SQL queries received over PostgreSQL wire protocol
 - Stores/retrieves data from disk
 - Sends query results back to FastAPI over TCP

2.2 Logical architecture

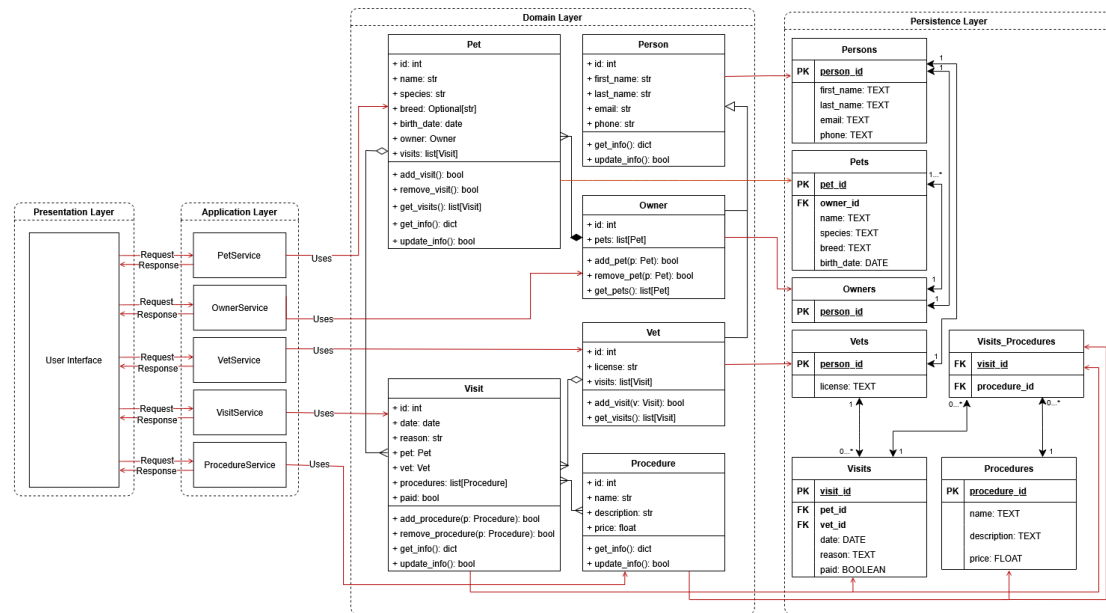


Figure 3: MiniORM logical architecture diagram

3 Implemented patterns

3.1 Strategy

The Strategy pattern allows defining a family of algorithms or implementation methods and making them interchangeable at runtime. In the context of database inheritance, this pattern is used to manage the three main strategies for mapping class hierarchies to tables:

Single Table Inheritance

All fields from all classes in the hierarchy are stored in a single table. A discriminator column identifies the specific class to which each record belongs.

Problem

When data is distributed across multiple tables, querying can become complex and often requires costly JOIN operations.

Solution

With Single Table Inheritance, all data is stored in one table, which simplifies querying and improves performance. Queries are faster, and inserting new records requires only a single insert operation.

Disadvantages

This approach results in a sparse table where many columns contain NULL values for records of types that do not utilize those specific fields.

Class Table Inheritance

Each class in the hierarchy has its own table, and tables are linked by primary keys to represent inheritance.

Problem

Relational databases do not support class inheritance directly, so storing fields from a base class and its subclasses in a single table can be inefficient or unclear.

Solution

In the CTI pattern, each class has a separate table. The base class table contains common fields, and each subclass table contains fields specific to that class plus a primary key linking to the base table. The ORM joins the tables when retrieving objects, allowing the application to work with a full object hierarchy.

Disadvantages

Retrieving a single subclass instance requires multiple JOIN operations across the hierarchy, which significantly impacts query performance.

Concrete Table Inheritance

Each type is stored in its own table, containing both common attributes and attributes specific to that type.

Problem

Querying data across multiple tables can be complex, while storing all data in a single table may lead to other issues, such as excessive table size and reduced readability.

Solution

Each type is assigned its own independent table. This approach allows tables to be optimized individually and keeps the schema clearer and easier to maintain.

Disadvantages

This strategy leads to significant data redundancy and schema duplication across multiple tables, making it difficult to perform mass updates on common fields.

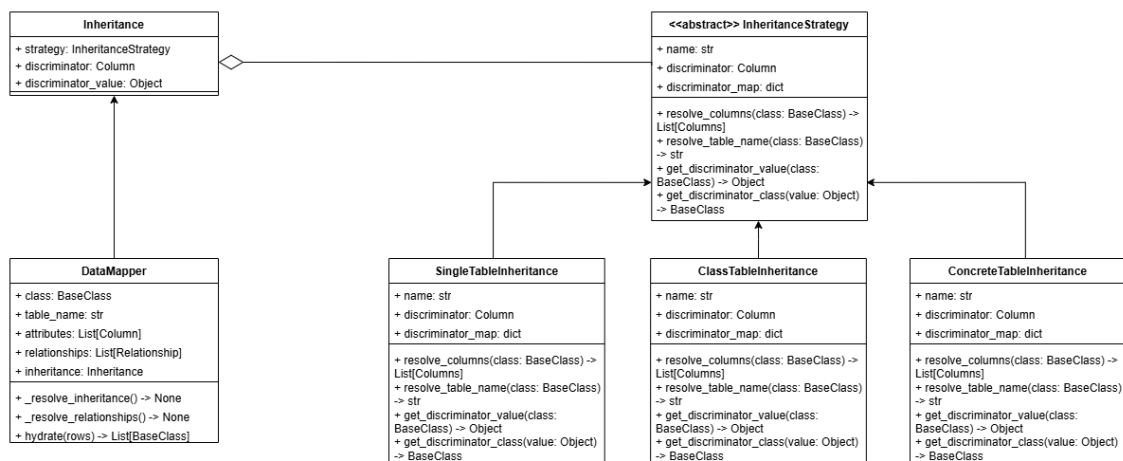


Figure 4: Strategy pattern

3.2 Unit of Work

Coordinates the writing out of changes to a database by maintaining a list of objects affected by a business transaction and managing their synchronization.

Problem

When an application modifies multiple related objects (e.g., creating a Visit, adding several Procedures, and updating a Pet's history), making individual database calls for each change is inefficient and can lead to data inconsistency if one operation fails while others succeed.

Solution

Implement a central Session using Unit of Work that tracks every object created, updated, or deleted during a specific task. The application registers changes in the Unit of Work instead of interacting with the database directly. When the transaction is complete, the Unit of Work calculates the necessary changes, resolves dependencies, and executes all required SQL commands in a single, atomic operation.

Disadvantages

The pattern increases memory consumption as it maintains references to all loaded objects within the Identity Map, which can lead to performance degradation in long-lived sessions. Additionally, since database interactions are deferred, debugging becomes more complex as SQL exceptions may occur during the final commit rather than at the point of data modification.

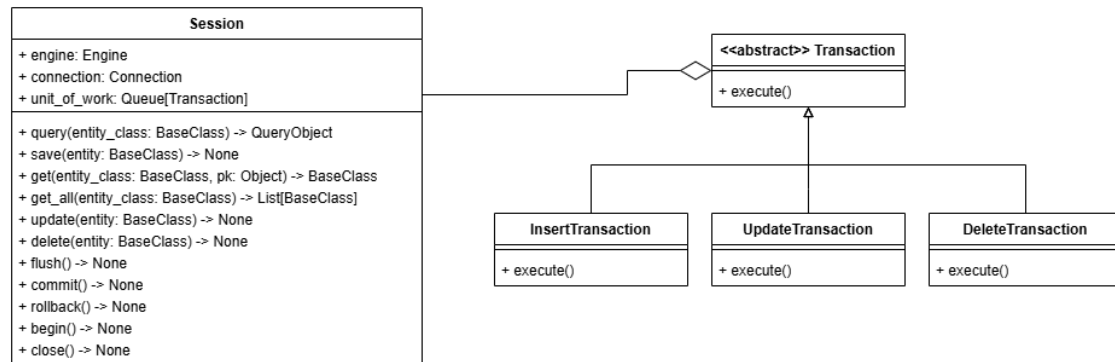


Figure 5: Unit of Work pattern

3.3 Data Mapper

A layer of software that separates the in-memory objects from the database, moving data between them while keeping them independent of each other. This promotes Persistence Ignorance, where domain entities (like Pet) remain focused solely on business logic, unaware of the underlying database schema or storage mechanism.

Problem

Directly coupling business logic with database access leads to 'fat models' that violate the Single Responsibility Principle. It also creates a rigid structure where any database change forces a rewrite of the business logic, exacerbating the object-relational impedance mismatch.

Solution

Implement a specialized Metadata Mapping layer (the Mapper class). Instead of domain objects performing SQL, the Mapper uses registered metadata to translate object states into relational commands. By utilizing an Inheritance Strategy, the mapper can also bridge the gap between complex object hierarchies and flat relational tables, ensuring that the domain model and database schema can evolve independently.

Disadvantages

The primary drawback of the Data Mapper is the significantly increased architectural complexity, as it requires a sophisticated metadata registry to track object-database mappings.

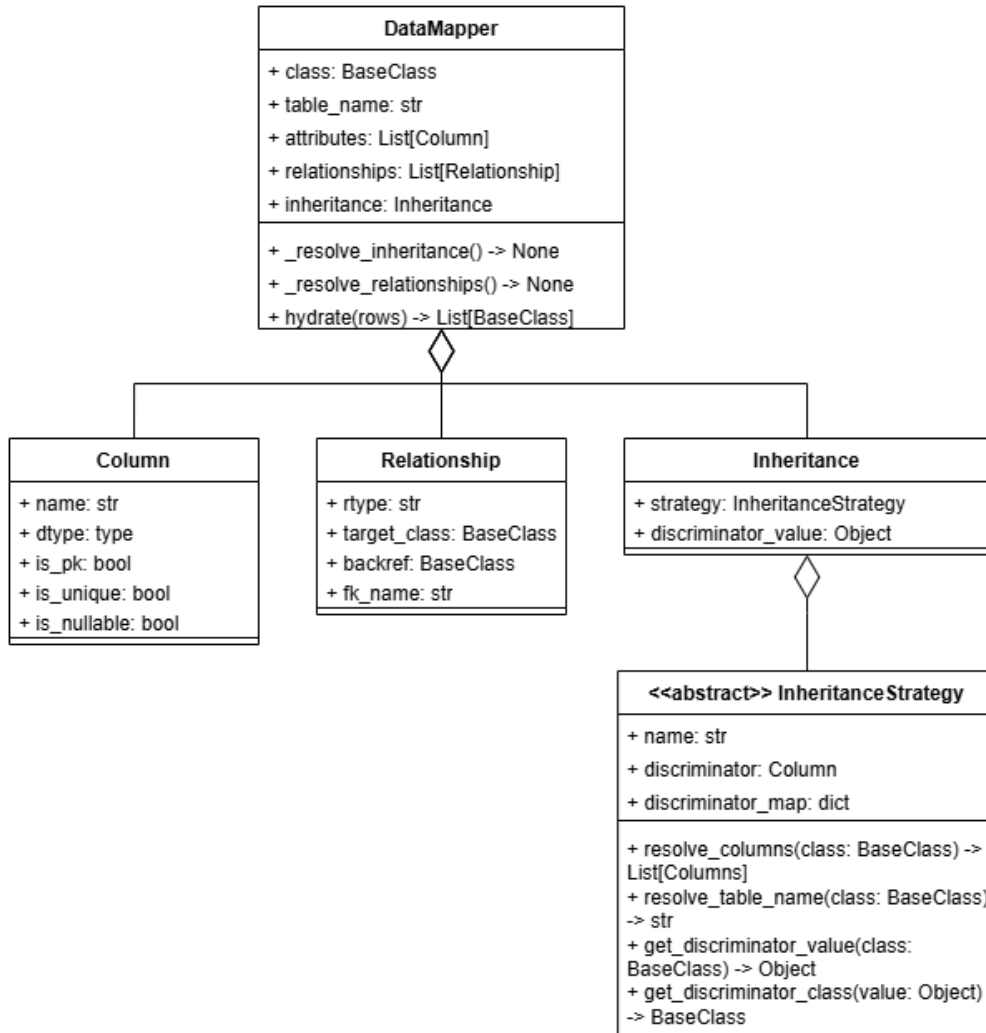


Figure 6: Data Mapper pattern

3.4 Identity Map

Ensures that each database record is loaded only once by keeping a map of all objects retrieved from the database during a single session. It acts as a gatekeeper that intercepts every request for an object and returns the existing instance if it has already been loaded.

Problem

Without an Identity Map, an application might load multiple independent instances of the same database record into memory. This leads to data inconsistency, as modifying one instance would not update the others, potentially resulting in conflicting updates and excessive memory consumption from redundant objects.

Solution

Implement a lookup table (the Identity Map) within the Session that stores objects using their class type and primary key as a unique identifier. Every query first checks this map; if the object exists, the system returns the cached instance. This ensures object identity and referential integrity across the entire business transaction.

Disadvantages

The pattern can lead to "stale data" issues, where an object in the map becomes out of sync if the underlying database is updated by an external process or a different session. Additionally, it places the burden of memory management on the developer, as the map grows with every loaded object, potentially requiring manual session clearing to prevent memory leaks in large-scale operations.

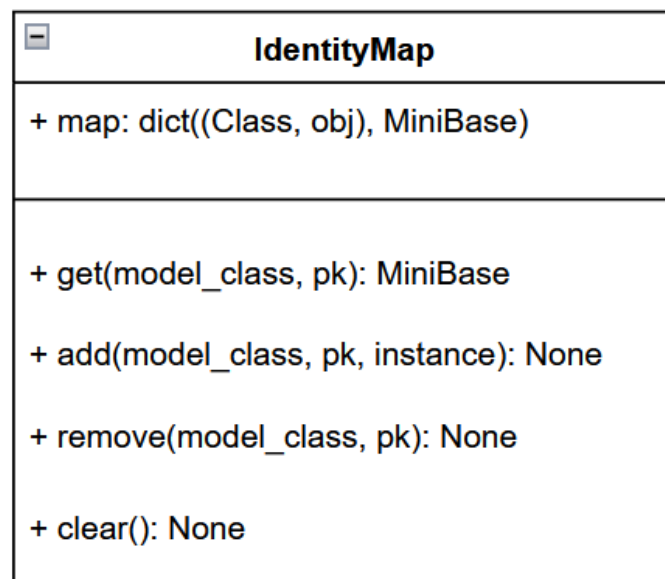


Figure 7: Identity Map pattern

3.5 Association Table

A specialized database table used to implement many-to-many relationships by linking two primary entities. It consists of foreign key columns pointing to both related tables, effectively decomposing a complex relationship into two simpler one-to-many associations.

Problem

Relational databases cannot natively represent many-to-many relationships. Without an intermediate structure, linking multiple records from both sides would require redundant data storage.

Solution

Implement a hidden Association Table managed entirely by the ORM metadata. The system automatically handles the mapping of these junction tables, performing necessary joins and link management behind the scenes. This allows domain entities to expose direct collections (like a list of Procedures in a Visit) while the underlying engine transparently handles the insertion and deletion of rows in the intermediate table.

Disadvantages

Association tables introduce additional JOIN operations, which can increase query execution time and complexity. They also create extra storage overhead and require careful cascading logic to ensure that

orphan records are not left behind in the junction table when one of the primary entities is deleted.

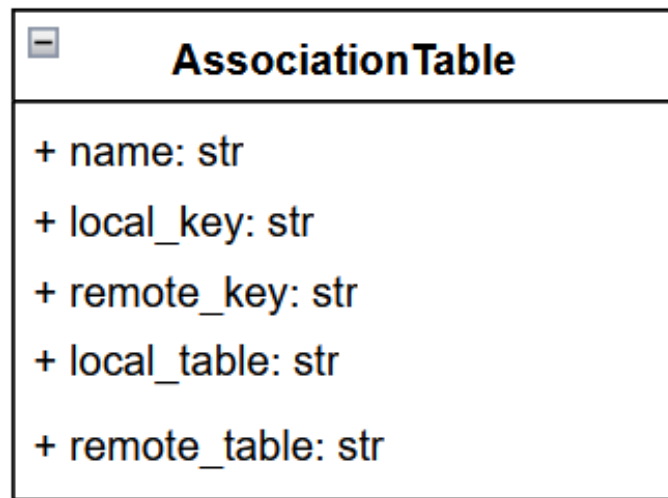


Figure 8: Association Table pattern